Colaberry Agent Platform Discovery Brief

The Colaberry platform runs a fleet of AI agents that report their own health. Much of that health is never measured, and the person accountable for the fleet cannot see it.

As of 27 Oct 2026 **Prepared by** Quincy Nkwain Ninying, Forward Deployed Engineer **Accountable** K.D. **Governance** A.M.

Basis five discovery stories, all verified; platform code at `c99021d` to `bb1d3c3`

242	180	33	0	23 / 48
agents in the platform's registry seed	scheduled timers: 111 in the main scheduler, 69 more through the cron wrapper	timers with no agent record, so no health surface sees them	agent surfaces the accountable manager can open	of this domain's capabilities reserved to a person at every level

THE PROBLEM

A quiet dashboard cannot be told apart from a blind one

The platform's agents run on timers. Each run is meant to leave a record, and the platform's monitors read those records to say whether the fleet is healthy. Where a record is missing, the monitors do not say "unknown": they say nothing, or they say healthy.

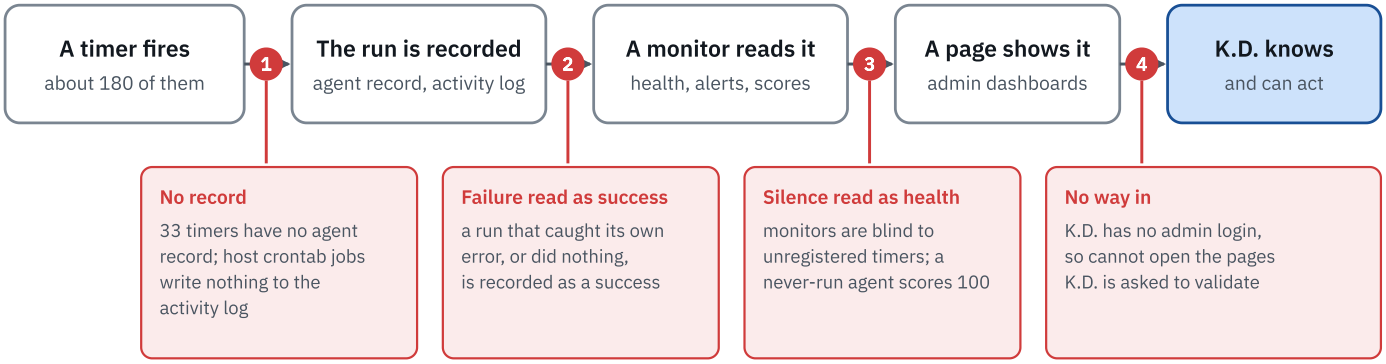
So the question a manager most needs answered, "is anything broken right now?", cannot be answered with confidence. A quiet result may mean all is well, or it may mean nothing was watching.

The manager accountable for this domain, K.D., has no admin login, so K.D. cannot open any of the surfaces K.D. is being asked to validate. The platform's own records mark the domain as unable to reach its surface.

WHERE THE SIGNAL IS LOST

Four points between a timer firing and the manager knowing

Each red marker is a place where a failure can leave no trace, or read as health.



WHO HAS THE PROBLEM

Four groups, each needing something different

K.D., accountable manager

Owns Platform Automation & Reliability and approves every agent action in production.

NEEDS

to see the fleet's real state without an admin login, and evidence to approve against

A.M., governance

Approves tools and pull requests, and reviews findings.

NEEDS

findings that hold up under a line-by-line check, labelled by risk

Platform on-call admin

Responds when something breaks.

NEEDS

to know which switch actually stops an agent, and which jobs leave no trace

Learners and leads

Receive what the agents send and decide.

NEEDS

agents that do not double-send, lose records, or close real problems as fixed

THE EVIDENCE

The blind spots, as the platform's own records show them

Gathered during discovery from the platform's code and its own capability catalogue, before any of Ellis was built.

Cron alerting cannot see some jobs

Structurally blind to unregistered, disabled and phantom timers.

`platform.json:1875`

A never-run agent scores 100

Agent health scoring gives a perfect score to an agent with no runs.

`platform.json:1926`

Untracked jobs leave no trace

Incident triage has nothing to follow, and the cron path records no error class.

`platform.json:2669`

Host jobs write nothing

Jobs scheduled directly on the host server leave no platform record when they fail.

`A.M., from the host crontab, 22 Oct 2026`

Approvals decided by a timer

106 approval records name a 4.25-hour timer as their only decider.

`promotionCriteria.test.ts:591`

No runbook, no agent of its own

No reliability runbook exists, and the domain had no agent record.

`agentDomainRegistry.ts:232`

WHAT IT ALREADY COSTS

Building Ellis confirmed the problem is not theoretical

From the Colaberry Platform Reliability Audit: 70 defects, 22 of them live risks. Three examples:

A duplicate email to the warmest lead

A slow batch could send the same message twice, to the lead who had just replied. Fixed by A.M. on receipt.

Decision history deleted unarchived

A cleanup could permanently delete rows it never copied, and record the run as healthy.

A fix announced that never happened

A bug-fix agent tells reviewers "fix applied" and moves the ticket to review, with nothing changed.

ASSUMPTIONS REGISTER

What discovery settled, and what is still open

Each resolved item names who settled it and when. Open items are stated, not guessed.

ASSUMPTION	STATUS	HOW IT WAS SETTLED, OR WHAT WOULD SETTLE IT
This domain needs an agentic system with human checkpoints, not a chatbot or a multi-agent system	● Resolved	K.D. approved version 3 on 22 Oct 2026, argued from the domain's figures `discovery story 1`
Some work moves to the agent, some is shared, and 23 decisions stay human	● Resolved	K.D. approved the split on 21 Oct 2026 `discovery story 3`

ASSUMPTION	STATUS	HOW IT WAS SETTLED, OR WHAT WOULD SETTLE IT
Logging coverage comes before monitoring, because monitors read the same incomplete records	Resolved	K.D. asked to proceed with the three tasks in this order on 26 Oct 2026 discovery story 5
Build with what exists and demo first, rather than wait for every document's sign-off	Resolved	K.D., 26 Oct 2026; A.M. refines after a demo
A check with no evidence must say "unknown", never healthy	Resolved	A.M. asked for the rule in writing, and accepted it no-evidence-is-not-health.md
The manager can be reached through an admin login, or another route until then	Open	An admin account for K.D., or a written briefing as the route in; a governance decision
Which "DA" responsibilities transfer: Data Analyst duties, or the Data Analytics student processes	Open	No document lists either; A.M. to confirm the reading, K.D. to list the duties
What "good" is in numbers: missed incidents and false alarms accepted, diagnoses right out of how many	Open	The manager interview's questions 6 and 7, not yet answered discovery story 4
What would switch the agent off at once, and who can while K.D. has no login	Open	The manager interview's question 8 discovery story 4

WHAT SUCCESS LOOKS LIKE

Checked by a person against a known answer, never marked done

OUTCOME	THE EVIDENCE THAT WOULD SHOW IT
Every scheduled job is accounted for	A list of which jobs are recorded and which are not, checked against the platform's own baseline and A.M.'s host crontab listing
No blind spot reads as healthy	For each check, a test that removes its signal and shows the answer is "unknown", not healthy
Monitoring K.D. can trust	Over a past window, every incident K.D. recorded is flagged, and K.D. confirms the extra flags are real or explained
Diagnoses that cite their evidence	On past failures with a known cause, the cause is named with the trace or log line
K.D. informed without an admin login	A sourced weekly briefing for four weeks, which K.D. confirms replaces the manual data pull

SCOPE

What this work does, and does not, take on

In scope

- This domain's 48 capabilities, built one at a time, each starting read-only
- Reading platform records and code, and reporting with evidence

Out of scope

- Changing the live platform: merging, deploying, or switching agents on or off
- Handing work to agents in other domains; nothing in this domain depends on one

- Repairs and reproductions on a sandbox copy only
- Reporting platform defects found along the way

- Deciding anything reserved to a person, at any level
- Fixing defects in the platform's own code; those go to its owners

SOURCES

- Discovery stories 1 to 5: system class, agent anatomy, role and DA transition, manager interview, three proposed tasks
- Colaberry Platform Reliability Audit: the 70 defects and their review outcomes
- No evidence is not health: the reporting rule
- Colaberry platform repository: every cited file and line, at the commit named in each discovery story